Fashnix

Team

Name	ID	Major
Rashed Abdulrazaq Boss	202331018	Information Security
Ahmed Mohammed AlBlooshi	202332415	Information Security
Mansour Khaled Alalawi	202111750	Information Security
Fares Al Ahbabi	202308052	Computer Science
Abdulrahman	700044029	Computer Science

1. Project Overview

Fashnix is an AI-powered outfit generation and marketing platform built for UAE SME fashion retailers. It lets a retailer upload their product catalog and automatically generate styled outfit combinations and social-media-ready captions using AI, along with analytics on usage and a branded export/dashboard experience.

2. Technology Stack

- Frontend & Backend: Next.js (App Router) — single codebase for both the UI and the server-side API routes
- Database & Auth: Supabase (Postgres database, Authentication, Row-Level Security)
- AI: Claude API (Anthropic) — outfit generation and caption generation
- Hosting & Deployment: Vercel (auto-deploys on push to GitHub main branch)
- Transactional Email: Resend (connected as custom SMTP provider for Supabase Auth emails)
- Source Control: GitHub — repository sima-boss/modamind

Live Production URL: https://modamind-xi.vercel.app

Demo account (email): demo@modamind.com

Demo account (password): 12345678

The demo account is pre-seeded with 93 products, sample billing history, and usage data for presentation purposes.

3. Development History — What Was Built, Step by Step

Step 1 — Authentication

- Real Supabase email/password authentication: signup (name, business name, strong password requirement), email verification, login, forgot/reset password, logout
- Profiles table with an auto-creation trigger on signup
- Default-deny middleware — every route is blocked unless explicitly allowed, rather than relying on each page to check authentication individually
- Open-redirect protection on the login/redirect flow
- All AI generation routes return 401 Unauthorized if called without a valid session
- Security audit finding: Row-Level Security (RLS) was found disabled on 5 live database tables — enabled, and 53 orphaned product records were backfilled to the correct owners
- Mobile navigation (hamburger menu) fixed for phone-width screens

Step 2 — Subscription Plans & Demo Checkout

- Three subscription tiers: Basic (149 AED/month), Standard (349 AED/month), Premium (799 AED/month)
- /choose-plan, /checkout (demo mode — no real payment processor), /billing pages
- Billing actions supported: upgrade (pay the difference immediately), downgrade (scheduled at the end of the billing period), cancel, and a "Simulate renewal" demo tool
- Billing history log
- All billing operations run as atomic Postgres functions restricted to the service_role, so they cannot be triggered directly by a client

Step 3 — Usage Limits & Credits

- Monthly outfit-generation and caption-generation limits enforced server-side (not just in the UI)
- Atomic credit deduction using database row locks, preventing race conditions (double-spending credits)
- Automatic refund of a credit if the AI generation call fails, handled inside the same server request (no separate client-callable refund endpoint, which would be an abuse vector)
- Product storage limit enforced at the database level via a trigger, not just application logic
- usage_events log table for auditing
- Paid top-up credit packs

Step 3.5 — Credits Display

- Unified "extra credits" wallet usable for either outfit or caption generation (1 credit each)
- Top-up pack pricing: 5 credits = 5 AED, 50 credits = 40 AED, 200 credits = 140 AED
- Usage bars capped visually at the plan limit, with a separate line for extra/top-up credits

Steps 4–5 — Analytics, Brand Kit, Plan Gating, Google Login

During development, an audit found a substantial batch of already-built work that had not yet been committed or deployed. Once found, it was committed, pushed, and deployed:

- Analytics dashboard with charts: outfits generated, captions generated, generation volume over time, usage split, busiest day
- Brand Kit feature (logo and brand color upload)
- Locked-feature / plan-gating components (features visibly locked for lower-tier plans, matching the plan each user is on)
- Google OAuth login ("Continue with Google"), fully integrated alongside email/password auth
- Six new database migrations covering: credit-system naming, outfit content language/format, brand kit schema, analytics functions, demo analytics seed data, and new-user OAuth handling

Final Integration & Deployment

- Custom SMTP connected via Resend, replacing Supabase's rate-limited default mailer, so confirmation and password-reset emails send reliably and quickly
- Feature branch merged into the main branch and deployed to Production on Vercel (19 October 2026)
- Full team testing pass completed on the live Production site with no critical issues found

4. Security Measures Implemented

As an Information Security-focused senior project, the following security measures are notable for the report:

- Row-Level Security (RLS) audit and fix across 5 core database tables, ensuring users can only access their own data at the database level, not just the application level
- Default-deny authentication middleware (deny by default, explicitly allow, rather than allow by default)
- Open-redirect sanitization, including protection against a backslash-based bypass technique
- All AI generation API routes reject unauthenticated requests (HTTP 401)
- Sensitive database functions (billing, credit deduction) use SECURITY DEFINER with an empty search_path and execution restricted to the service_role, preventing privilege escalation
- Atomic credit deduction with row-level locking prevents double-spending of generation credits

- No client-callable refund endpoint — refunds only happen server-side inside the same failed request, closing a potential abuse path
- Product storage limits enforced at the database layer via triggers, not just client-side checks
- Secret key rotation performed after a Supabase service-role key was accidentally exposed; replaced with a new key and the old one revoked
- API keys scoped with least-privilege access and expiry where supported

5. System Architecture

Fashnix is built on Next.js (App Router) for both frontend and backend, deployed on Vercel with GitHub-triggered auto-deployment. The frontend renders pages for the dashboard, plan selection, checkout, billing, analytics, and brand kit. These call internal API routes (/api/auth/*, /api/outfits/*, /api/billing/*, /api/analytics/*), which are the only layer allowed to talk to Supabase and the Claude API — no secret keys are ever exposed to the browser.

All requests pass through middleware that enforces a default-deny auth policy, sanitizes redirect URLs to prevent open-redirect attacks, and refreshes the Supabase session.

Supabase provides authentication (email/password and OAuth), the Postgres database, and Row-Level Security (RLS) on all core tables, so each user can only read/write their own data at the database level. Sensitive operations (billing, credit deduction) run as SECURITY DEFINER Postgres functions restricted to the service_role, preventing privilege escalation from client requests.

The Claude API (Anthropic) handles outfit and caption generation, called only from server-side API routes with usage limits and credit deduction enforced atomically before each request.

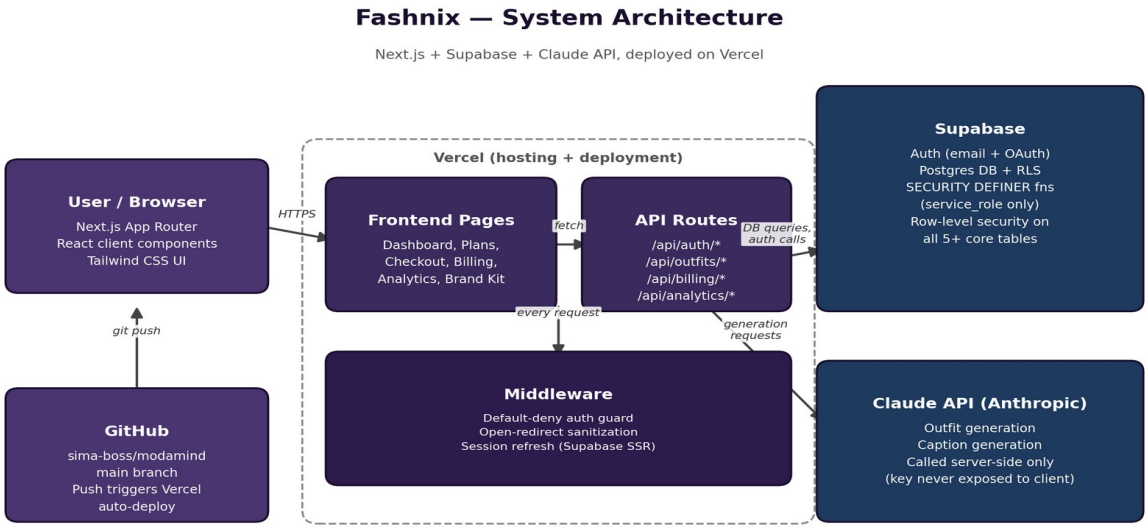


Figure 1 — Fashnix system architecture

6. Testing

The platform was tested by all five team members on the live Production deployment, following the final merge to main on 19 October 2026. Each member tested independently on their own device.

#	Feature Tested	Result
1	Sign up (new account creation)	Pass
2	Email confirmation flow	Pass
3	Login / logout	Pass
4	Forgot password / reset password	Pass
5	Dashboard load and data accuracy	Pass
6	Plan selection and demo checkout	Pass

7	Outfit generation (Claude API)	Pass
8	Caption generation (Claude API)	Pass
9	Analytics dashboard (charts)	Pass
10	Brand Kit upload	Pass
11	Export functionality	Pass
12	Mobile responsiveness	Pass
13	Billing page (plan, usage display)	Pass

All five testers confirmed the core user flows worked as expected, with no critical bugs found.

7. Implementation Scope — What Was Deliberately Not Built

All core features of Fashnix are implemented and live in Production: authentication (email/password + Google OAuth), subscription plans with demo checkout, usage-based credit system, AI outfit and caption generation, analytics dashboard, brand kit, and export functionality. Custom SMTP (Resend) handles all transactional email reliably.

One feature was deliberately scoped out of this release: team/workspace support (multiple users sharing one account/subscription). It was deprioritized because it requires a significant rewrite of the Row-Level Security (RLS) policies — the highest-risk, lowest-visibility change relative to the project's core AI functionality — and the team judged it better to ship a stable, fully-tested product than risk breaking a working system this close to submission.

Recommendation: list team/workspace support as a recommended future enhancement in Chapter 6 (Conclusion and Future Recommendations).

8. Reference Links

- Live site: https://modamind-xi.vercel.app
- GitHub repository: https://github.com/sima-boss/modamind
- Demo login: demo@modamind.com / 12345678